



C#

Level 1

Sum of Matrix



C#

Level 1

Sum of Matrix

Definition

Numeric Data Types

 Numeric data types are used to store whole numbers and numbers with decimal parts.

 C# provides integer, floating-point, and decimal numeric types with different sizes, ranges, and precision.

Explanation

Integer Types

-  ``byte`` → 8-bit unsigned → 0 to 255.
-  ``sbyte`` → 8-bit signed → -128 to 127.
-  ``short`` → 16-bit signed → -32,768 to 32,767.
-  ``ushort`` → 16-bit unsigned → 0 to 65,535.
-  ``int`` → 32-bit signed → -2,147,483,648 to 2,147,483,647.
-  ``uint`` → 32-bit unsigned → 0 to 4,294,967,295.
-  ``long`` → 64-bit signed → very large positive and negative integers.
-  ``ulong`` → 64-bit unsigned → very large non-negative integers.

Floating-Point Types

-  ``float`` → 32-bit single precision.
-  ``double`` → 64-bit double precision.
-  ``decimal`` → 128-bit decimal type with high precision and a smaller range than ``double``.
-  ``decimal`` is commonly suitable for financial and monetary calculations.

Minimum and Maximum Values

-  Each numeric type provides ``MinValue`` and ``MaxValue``.
-  They can be used to check the smallest and largest values supported by the type.

Procedures

1. Choose a numeric type according to the required range and precision.
2. Declare the variable.
3. Assign a value within the supported range.
4. Use `MinValue` and `MaxValue` when checking the type's limits.

💡 Example

```
Console.WriteLine(int.MinValue);  
Console.WriteLine(int.MaxValue);
```

➡ Result

```
-2147483648  
2147483647
```

Note

- ◆ Signed types can store negative and positive values.
- ◆ Unsigned types can store zero and positive values only.
- ◆ Larger types generally provide a larger range, while floating-point types differ mainly in precision and range.

⊘ Warning

- ◆ A value outside the type's range cannot be assigned directly.

Example:

```
byte b = 255;    // ✓
```

```
byte x = 256;    // ✗
```

- ◆ Use the correct suffix when required:

```
float f = 10.5f;
```

```
long l = 100000L;
```

```
decimal m = 10.5m;
```



Comparison

▨ Integer Types

- ✓ Store whole numbers.
- ✓ Include signed and unsigned types.
- ✓ Best when no decimal part is required.

▤ Floating-Point / Decimal Types

- ✓ Store numbers with decimal parts.
- ✓ `float` → lower precision and smaller size.
- ✓ `double` → higher precision and larger range.
- ✓ `decimal` → high decimal precision, commonly used for financial calculations.

✦ Summary

- ✓ Numeric types store whole numbers or decimal values.
- ✓ Integer types range from 8-bit to 64-bit.
- ✓ Signed types support negative values.
- ✓ Unsigned types support zero and positive values only.
- ✓ ``float``, ``double``, and ``decimal`` support decimal values.
- ✓ ``MinValue`` and ``MaxValue`` show the limits of a numeric type.
- ✓ ``e`` / ``E`` represents scientific notation.
- ✓ ``0x`` represents hexadecimal.
- ✓ ``0b`` represents binary.

Thank you 😊